

## INTRODUCTION

This first assignment focuses on parallelizing a spatio-temporal co-clustering algorithm using Message Passing Interface (MPI). The goal is to divide the computation across multiple nodes of the DAS6 cluster so that large datasets can be processed faster. By distributing the work among multiple processes and coordinating them through MPI communication, the parallel program aims to achieve better performance while producing results similar to the original serial implementation.

The MPI program divides the work by sharing the columns of the matrix among processes. The function `get_col_partition` helps to find out how many columns each process should get, this way the columns are divided evenly as possible. Each process gets only its part of the columns; however, it still keeps all rows because the row labels are needed to calculate distances. The process with rank 0 reads the matrix first. It rearranges the matrix so that columns are stored together. Then it sends the block of columns to each process. Then, it uses `MPI_Scatterv` to send the columns along with the matching column labels. After that every process works on its local matrix and local column labels. The row labels are the same for all processes. The MPI communication helps to keep all processes synchronized. In every iteration the process with rank 0 calculates the cluster averages. It updates the row labels just like in the serial implementation. Then it sends the row labels and cluster averages to all processes- it uses `MPI_Bcast` for this. Each process then updates the labels for its own columns independently. The number of columns that changed and the distance values are sent back to rank 0 using `MPI_Reduce`. The updated column labels from all processes are gathered back using `MPI_Gatherv`. Finally, process with rank 0 checks if the algorithm has stabilized. It broadcasts this information so that all processes stop the loop at the time.

### Normalized Mutual Information Table

| Dataset      | Clusters | No. of Processes | NMI     |
|--------------|----------|------------------|---------|
| 1. Small (s) |          |                  |         |
|              | 3*20     | 2                | 1.00000 |
|              | 5*100    | 2                | 1.00000 |
| 2. Medium(m) |          |                  |         |
|              | 3*20     | 2                | 1.00000 |
|              | 5*100    | 2                | 1.00000 |
| 3. Large (l) |          |                  |         |
|              | 3*20     | 2                | 1.00000 |
|              | 5*100    | 2                | 1.00000 |

## Performance Test with XL Dataset1

| Dataset          | Clusters | Processes | Runtime (s) | Speedup |
|------------------|----------|-----------|-------------|---------|
| Extra Large (XL) |          |           |             |         |
|                  | 3*20     | 1         | 1045.28     | 1.0     |
|                  | 3*20     | 2         | 647.232     | 1.61    |
|                  | 3*20     | 4         | 532.371     | 1.96    |
|                  | 3*20     | 6         | 403.331     | 2.58    |
|                  | 3*20     | 8         | 347.332     | 3.01    |

## Speedup Graph

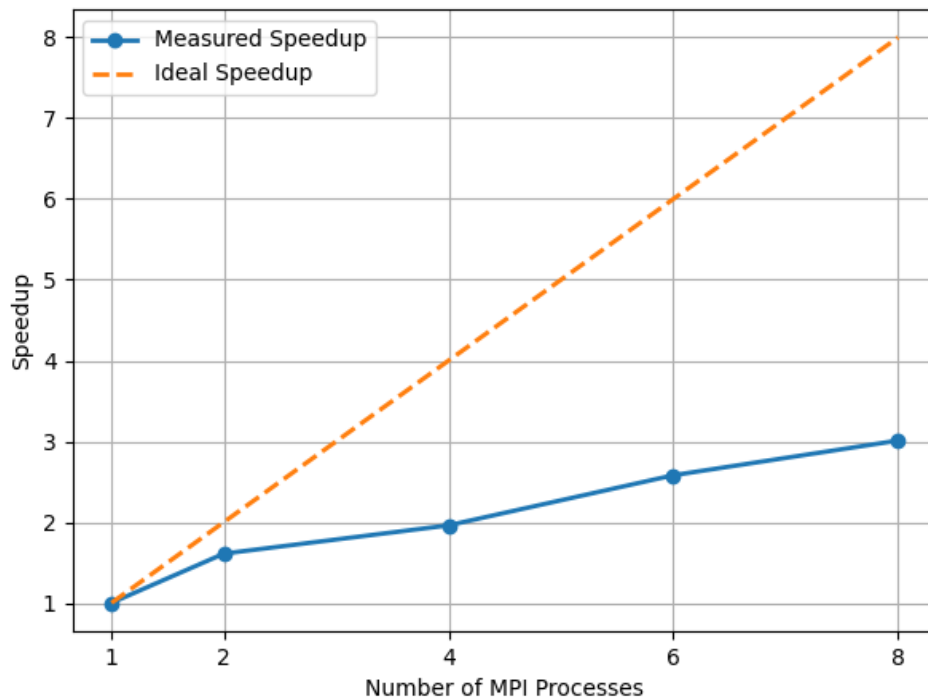


Figure 1 Speed Up graph

The above figure shows the scaling of the MPI implementation on the XL dataset. The measured speedup increases as the number of processes increases, but it does not reach the ideal linear speedup due to communication and coordination between MPI processes.